WASP Software Engineering Course Module 2026 – Assignment

Aleix Nieto Juscafresa
Uppsala University & WASP Graduate School

September 1, 2026

1 Introduction

I am a PhD student at Uppsala University, supervised by Jens Sjölund, and my project is part of AID4BC, a joint WASP and DDLS NEST initiative on multimodal machine learning for breast cancer detection. Population-based mammography screening reduces breast cancer mortality, but it operates under hard constraints. Radiologist time is scarce, the majority of examinations are normal, and the cancers that matter most are often subtle. Machine learning is a natural candidate for detection and triage support, yet progress is limited less by model architectures than by data and evaluation. Clinical images are privacy sensitive and hard to share, labels are weak and late (definitive ground truth may require biopsy or years of follow-up for interval cancers), and the rare cases that matter most are, by definition, underrepresented.

My current work attacks this bottleneck through virtual clinical trials (VCTs), in which digital breast phantoms, imaging physics, and eventually reader models are simulated in silico, so that imaging systems and algorithms can be evaluated on populations with exactly known ground truth. Concretely, I am working on optimizing the parameters of procedurally generated breast phantoms based on Perlin noise. The generator is controlled by a parameter vector θ, and a differentiable measurement model maps phantoms to synthetic mammograms, so that θ can be fitted by gradient-based optimization until simulated images match the statistics of clinical real mammography data. The first milestone is a parameter-recovery self-consistency study: generate images from a known θ^*, re-estimate it from those images alone, and require the pipeline to close the loop. Longer term, the phantom population is intended to complement scarce clinical data for training and, above all, for stress-testing multimodal detection models.

Seen through the lens of this course, my research output is software. It is a simulation chain, a set of data pipelines over heterogeneous DICOM archives, and training and evaluation harnesses, and its most dangerous failures are silent. Images from the wrong processing domain, a subtly mis-specified noise model, or leakage between data splits do not crash anything; they quietly invalidate results. This essay therefore looks at the course material from a specific perspective: software engineering for AI is not just extra work added to my research, but something that can actually have an impact on it. Sections 2 and 3 relate principles from the lectures and guest lectures to this setting, Section 4 discusses the roles around AI systems, Section 5 analyzes two papers, and Section 6 accounts for the sources I used.

2 Lecture principles

2.1 The test oracle problem and metamorphic testing

Lecture 2 on quality assurance and testing explained the difference between verification and validation. One point I initially disagreed with was Kästner's idea that testing ML components is mostly validation, since learned models usually have no clear specification to verify against [1]. This leads to the *test oracle problem*: for many inputs, we simply do not know the correct output. The lecture suggested metamorphic testing as a solution, where we test whether outputs change in reasonable ways when we modify the inputs [7].

This reframed my own evaluation practice in two ways. First, screening data makes the oracle problem concrete. Per-image truth is expensive and delayed (biopsy, or interval follow-up), so checks that need no labels are precious, and metamorphic relations provide them. A detection model should be invariant to anatomically irrelevant transformations such as left-right mirroring, and its performance should degrade monotonically as image quality degrades. My differentiable measurement model makes the second kind of relation executable, because I can render the same phantom at reduced dose or increased blur and assert monotone behavior of any model under test. Second, I now see the VCT pipeline as an oracle factory. Simulation produces images with exact, voxel-level ground truth, which turns fuzzy system questions (is this lesion detectable at this dose?) into runnable tests. Even my parameter-recovery study is a metamorphic test of the estimation software itself: generate from a known θ^*, re-estimate, and require $\hat{\theta} \approx \theta^*$. It consumes no external labels, yet a failing run localizes a defect. What I used to call an evaluation protocol, I now maintain as a test suite.

2.2 Hidden technical debt in machine learning systems

The opening lecture (L1) and the first discussion session, which centered on Sculley et al. [2], introduced the forms of technical debt specific to ML: entanglement summarized as CACE ("changing anything changes everything"), unstable and underutilized data dependencies, pipeline jungles, configuration debt, and training/serving skew, together with later evidence that the catalogue has aged well [7].

My pipeline exhibits each risk in miniature, and one of them nearly bit. Mammography DICOMs exist in two domains, FOR PROCESSING (raw detector-domain data) and FOR PRESENTATION (vendor-postprocessed data for display), and their image statistics differ fundamentally. Quantities my work relies on, such as the power-law exponent β of the noise power spectrum and its canonical validity range $\beta \in [2,3]$, are only meaningful in the right domain. Fitting phantom parameters against one domain while validating against the other is textbook training/serving skew: nothing crashes, the numbers are simply wrong. The fix was engineering rather than modeling. The image domain is now explicit, validated metadata asserted at every pipeline boundary instead of tribal knowledge in my head. CACE appeared when I reparameterized the Poisson noise model and the optimal texture parameters shifted with it; because components of θ are entangled with the physics model, the parameter-recovery study now runs as a regression gate after any change to the measurement model. Finally, my reliance on external archives (MBTST-DM) is data-dependency debt, so I pin dataset versions and DOIs for the same reason one locks package versions. The lecture's core lesson, that in ML systems the debt lives mostly outside the model code, matches my experience precisely.

3 Guest-lecture principles

3.1 Requirements live in the problem space and must be made checkable early

Julian Frattini's guest lecture on requirements engineering focused on the difference between the problem and solution spaces, and on defining requirements in terms of stakeholder needs rather than specific solutions. It also introduced the 10x rule, where fixing a requirements problem becomes roughly ten times more expensive at each later development stage. The RE4AI part showed how ML quality requirements such as robustness and fairness are often vague and difficult to turn into measurable requirements [7].

This is directly relevant to my project, where our main requirement was that synthetic mammograms should be *realistic*. We eventually replaced this vague idea with measurable criteria, such as valid noise properties, physically consistent blur and noise, and successful parameter recovery. We also removed a lesion-detection experiment that we could not properly support yet. This made me realize that defining requirements early can save a lot of work later. I also found the idea of misuse cases useful, since clinical AI systems need to consider foreseeable misuse, such as using a model outside its validated population or modality.

3.2 The semi-executable stack and hidden organizational gaps

Per Lenberg's guest lecture on the human aspects of AI focused on how organizations can be seen as a *semi-executable stack*. The lower layers, such as code and prompts, run on machines, while the upper layers, such as workflows, roles, and regulations, are carried out by people. He also emphasized that AI makes change more constant, exposes hidden problems in organizations, and does not necessarily make work better just because it reduces routine tasks [7].

I can see this clearly in AID4BC, where much of the difficulty is above the code. Models and pipelines are only one part of the system; annotation guidelines, screening workflows, PACS conventions, and regulations all depend on people. The confusion between FOR PROCESSING and FOR PRESENTATION in Section 2.2 is a good example of a hidden gap that only became obvious when we automated the data pipeline. I also realized that questions such as who defines the ground truth or who sets a model's operating threshold are organizational questions, not things that the model itself can solve. This made me see the many discussions in my project not as overhead, but as an important part of engineering the whole system.

4 Data scientists, software engineers, and AI engineers

The distinction, and whether I agree. The chapters describe data scientists as mainly focused on statistics, machine learning, and exploring data, often working in notebooks and mainly evaluating models based on accuracy. Software engineers, on the other hand, focus more on building products that meet user needs while also considering things like cost, usability, scalability, security, and maintainability. Kästner describes people who are comfortable with both areas as rare "unicorns" and argues that T-shaped people in interdisciplinary teams are a more realistic goal. I agree with this distinction as a general description, although it is clearly simplified. In academic research, the two roles often become one person. In my own work, I switch between exploratory notebook work and writing more robust pipeline code, with different expectations for each. I would also add the domain expert as an important role in my field, since radiologists and medical physicists help determine whether we are actually measuring the right thing.

The system-wide view in my research area. The idea that model quality is not the same as system quality is especially relevant to medical screening. A model with a better AUC could still make the overall screening process worse if it increases the workload for radiologists or reduces their trust in the system [5]. Safety is also a system-level property. A triage model is used together with double reading, arbitration, thresholds, and monitoring, so these parts matter as much as the model itself. My own research follows this idea through virtual clinical trials, where the whole chain from anatomy and image formation to reading is simulated instead of only evaluating individual model metrics. Even decisions about my generator, such as which noise exponent or image domain to use, are influenced by how the generated images will be used downstream.

Will the roles remain distinct? I think both convergence and specialization will continue. Tools such as MLOps platforms and foundation models make it easier for engineers to work with ML, while coding assistants make it easier for data scientists to write more engineering-oriented code. This makes shared knowledge increasingly important. Microsoft's early case study also found that many of the difficult problems were related to data, reuse, and interactions between components rather than just coding [6]. However, I do not think the roles will completely merge. Someone still needs to understand issues such as sampling bias, while someone else needs to design systems that are reliable and fail safely. I expect the boundary to become less strict, with more shared ownership of pipelines and evaluation rather than simple handovers between roles.

Where AI engineering fits. The chapters describe "AI engineering" as a term that is often used mainly for model pipelines, deployment, and MLOps [1]. In this course, I see it more broadly as software engineering for systems whose main components are learned from data rather than fully specified. This makes it more than just traditional software engineering applied to AI. Problems such as the test oracle problem, data and feedback-loop issues, and the difficulty of specifying the behavior of learned models give AI engineering some challenges that are specific to AI systems.

How foundation models, assistants, RAG, prompt pipelines, and agents change the boundaries. These technologies move the boundary from mainly training models towards composing and adapting them. Prompts, retrieval data, and evaluation pipelines become artefacts that need to be versioned and tested, bringing more software and data engineering into ML work. Coding assistants and agents also make it easier for people to build AI systems without deep ML knowledge, but someone still needs to be responsible for how those systems behave. In medical imaging, this responsibility is especially important because the final clinical validation still belongs to a human organization. For my own work, this makes the engineering practices from Section 2.1 more important rather than less important: even if an assistant generates the code, I still need a way to test whether it is correct.

5 Paper analysis

Both selected papers are full-length peer-reviewed conference papers. The first is from CAIN; the second falls under the 2026 course themes of LLMs for software engineering and agentic software engineering.

5.1 Foidl, Felderer, and Ramler, "Data Smells: Categories, Causes and Consequences, and Detection of Suspicious Data in AI-Based Systems" (CAIN 2022)

Core ideas and their SE importance. The paper transfers the code-smell idea to data. A *data smell* is a suspicious value or property in a dataset that does not stop the pipeline but indicates a latent quality issue likely to degrade an AI-based system later; a smell is not a defect, it is a reason to look closer. The authors contribute a definition, a catalogue of recurring smells grouped into believability, understandability, and consistency categories rooted in classical data-quality dimensions, an account of typical causes and of consequences for AI-based systems, and a discussion of detection, from rule-based checks to statistical analysis and tool support [3]. The SE importance follows from the premise of this whole course: in AI systems, data does much of the work that source code does elsewhere, yet it receives none of the review machinery. Smells provide a shared vocabulary and, crucially, an automation hook: they can be encoded as validation gates in pipelines and continuous integration, enabling early and cheap detection in the spirit of the 10x rule, and they complement model-level testing, which can reveal that quality dropped but rarely localizes the fault to the data that caused it.

Relation to my research. My raw material is DICOM data from different vendors and archives, so data quality problems are something I deal with regularly. I often find pixel statistics that suggest the wrong processing domain even when the metadata looks correct, as well as burned-in annotations, inconsistent laterality and view information, different bit depths, and duplicate patients that could cause data leakage between splits. The paper gave me useful names and categories for checks that I had previously been writing more or less ad hoc while preparing MBTST-DM inputs. This also connects closely to my phantom work. Synthetic data is clean by construction, so it gives me a controlled way to debug the pipeline without worrying about unknown problems in the input data. At the same time, the fitted θ and the measurement model give me an idea of what reasonable image statistics should look like. This means they can also be used to detect suspicious clinical data. For example, an estimated β outside its validity range could indicate that an image comes from the wrong processing domain or from an unexpected acquisition setting. In this way, the phantom pipeline is not only useful for generating data, but can also help identify data quality problems in real datasets.

Integration into a larger AI-intensive project. Consider a national screening-support platform: AI triage integrated with PACS across dozens of hospitals and heterogeneous scanner fleets, where AI is the core of the product. The paper's catalogue becomes the basis of an ingestion firewall. Every incoming study passes domain, metadata-consistency, and distributional checks; per-site smell dashboards surface systematic problems; onboarding a new hospital or a scanner upgrade requires passing the smell suite before its data may reach training or inference; and a spike in smell rates triggers incident response, since it usually means the upstream world changed. My WASP research fits this project as both a supplier of detection models and, more distinctively, as the source of reference statistics and synthetic regression data that give the firewall quantitative teeth.

Adaptation of my research. Three changes could make my project more closely follow the ideas in the paper in the long term. First, I could use the phantom generator to deliberately create problematic data, such as images with the wrong processing domain, clipped intensity ranges, or resampled grids. This would allow me to test whether data-quality checks actually detect these problems and how models behave when they are introduced, which is similar to mutation testing but applied to data. Second, I could turn the validity ranges from our manuscript into machine-checkable requirements instead of leaving them only as prose, making it easier for others to include them in their own validation pipelines. Third, I could make the parameter-recovery study a regular automated check rather than a one-time experiment. Running it continuously would provide an early warning if changes to the simulation or data pipeline introduced unexpected errors. Together, these changes would make the pipeline more testable and make data quality a continuous part of the development process rather than something checked only at the end.

5.2 Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" (ICLR 2024)

Core ideas and their SE importance. SWE-bench evaluates language models on real software maintenance. Each of its 2,294 tasks pairs a GitHub issue with the full repository state from one of twelve popular Python projects; the model must produce a patch, and success is judged by executing the project's own tests, requiring designated failing tests to pass while the remaining suite keeps passing [4]. Three properties matter. The tasks are realistic in scale (large codebases, multi-file edits, long context), grading is execution-based rather than text-similarity-based, and the construction pipeline can continually harvest new tasks from live repositories. The reported results were sobering, with the best models of the time resolving only a small percentage of issues under realistic retrieval. For software engineering the paper is important because it redefined how AI-for-code is measured, moving from isolated function synthesis toward system-level tasks where the test suite serves as the specification, an old SE idea given new load-bearing work. It also exposed how much engineering evaluation infrastructure itself needs; the later human-validated subset (SWE-bench Verified) exists because benchmark quality became a first-order concern.

Relation to my research. The relation is quite direct. AI coding assistants now help me write parts of my research software, while SWE-bench shows how reliable they are on realistic software engineering tasks. This supports my working rule that code generated by an assistant is a draft, and that it should only be accepted if it passes my own tests, with the parameter-recovery study providing a concrete oracle for detecting failures. The paper also connects to the main measurement problem in my research. Just as scores on curated coding tasks can make AI assistants look more capable than they are in practice, accuracy on enriched or single-site medical datasets can give an overly optimistic picture of screening performance. In both cases, the solution is to evaluate systems in conditions that are closer to their real use. SWE-bench does this through executable test suites, while virtual clinical trials do it through simulated physics and known lesions. For me, the main lesson is that good evaluation is often more important for trustworthiness than simply choosing a better model.

Integration into a larger AI-intensive project. Take the same national screening platform. Its non-model software, ingestion, preprocessing, monitoring, and integration code, will increasingly be maintained with agentic coding tools, and SWE-bench's design translates directly. The project would build an internal benchmark from its own issue tracker and regression tests to select, and continuously monitor, coding agents before they may open pull requests, requiring fail-to-pass demonstrations plus pass-to-pass non-regression on the medically critical suites, including the data-smell suite from the first paper. My research fits by supplying phantom-based tests that give those suites clinical meaning: an agent's refactoring of preprocessing code must preserve detection performance on simulated cases with known ground truth, otherwise the patch is rejected regardless of how clean it looks.

Adaptation of my research. Long term, I would restructure my codebase so it could join such a benchmark: issues written with reproduction steps, deterministic simulation seeds so that fail-to-pass is meaningful, and a test suite whose green and red states genuinely carry the specification. A natural extension is an execution-grounded benchmark for medical-imaging pipelines in which each task is a realistic maintenance change and success is defined by preserved θ-recovery and detection metrics on synthetic cases. That would support agentic tooling and, as a side effect, harden the science, since a pipeline that an agent can safely modify is a pipeline whose assumptions have finally been written down.

6 Evidence

The main sources I relied on, and how I used them, are the following.

- Lecture and discussion material of this module (Feldt's lectures and the guest lectures by Frattini and Lenberg) [7]. Kind: course material (slide decks and in-class discussion). Used for the framing and the principles in Sections 2 and 3.
- Kästner, *Machine Learning in Production*, chapters 1 and 2 [1]. Kind: book chapters (published by MIT Press and openly available online). Used as the basis for Section 4.
- Foidl, Felderer, and Ramler, CAIN 2022 [3]. Kind: peer-reviewed conference paper. Analyzed in Section 5.
- Jimenez et al., ICLR 2024 [4]. Kind: peer-reviewed conference paper (with released code and data). Analyzed in Section 5.
- Sculley et al., NeurIPS 2015 [2]. Kind: peer-reviewed conference paper grounded in production experience at Google. Read for the first discussion session and used in Section 2.2.
- Bernardi et al., KDD 2019 [5], and Amershi et al., ICSE-SEIP 2019 [6]. Kind: peer-reviewed industry-track papers. Used as supporting evidence in Section 4.

I place the most trust in the peer-reviewed papers that report primary evidence. Kästner's book is a single-author synthesis rather than primary research, but it is heavily sourced and widely used in teaching, so I trust it for framing while treating its role characterizations as deliberate simplifications. The lecture decks are curated secondary material; I trust them for the course's framing and checked load-bearing claims against the primary sources they cite. I used no blog posts or product pages as evidence.

References

[1] C. Kästner. *Machine Learning in Production: From Models to Products*. MIT Press, 2025. Openly available at `https://mlip-cmu.github.io/book/`.

[2] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 2503–2511, 2015.

[3] H. Foidl, M. Felderer, and R. Ramler. Data smells: Categories, causes and consequences, and detection of suspicious data in AI-based systems. In *Proceedings of the 1st International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2022.

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[5] L. Bernardi, T. Mavridis, and P. Estevez. 150 successful machine learning models: 6 lessons learned at Booking.com. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.

[6] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann. Software engineering for machine learning: A case study. In *Proceedings of the International Conference on Software Engineering, Software Engineering in Practice track (ICSE-SEIP)*, 2019.

[7] R. Feldt, J. Frattini, and P. Lenberg. Lecture and guest-lecture material, WASP "Software Engineering and Cloud Computing" course module, Chalmers University of Technology, June 1–2, 2026.